Лабораторная работа №8

Тема работы: «Разработка алгоритмов и программ обработки множеств»

1 Цель работы

Научить разрабатывать алгоритмы и программы обработки множеств.

2 Задание

1. Даны два слова. Проверьте, являются ли эти слова анаграммами друг друга?

```
#include <iostream>
#include <string>
#include <conio.h>
#include <algorithm>
#include <set>
using namespace std;


void task1();
void task2();
void task3();


int main() {
    setlocale(LC_ALL, "RU");

    cout << "ВЫБОР ЗАДАНИЯ" << endl;
    cout << "1. Проверить, являются ли слова анаграммами друг друга" << endl;
    cout << "2. Проверьте, есть ли баланс скобок в строке" << endl;
    cout << "3. Введите номер в строке и выясните какому слову по счёту в строке он принад-
лежит" << endl;

    int num_zad;

    while (true) {
        cout << "\nВведите номер задания от 1 до 3 (ESC для выхода): ";

        if (_getch() == 27) break;

        cin >> num_zad;
        cin.ignore();

        switch (num_zad) {
        case 1: task1(); break;
        case 2: task2(); break;
        case 3: task3(); break;
        default:
            cout << "Неверный номер задания!" << endl;
            break;
        }
    }

    return 0;
}


void task1() {
    cout << "ВЫПОЛНЯЕТСЯ 1 ЗАДАНИЕ\n";
    string str, str1;
```

```
    cout << "Введите строку1: ";
    getline(cin, str);
    cout << "Введите строку2: ";
    getline(cin, str1);

    multiset<char> set1, set2;

    for (char c : str) {
        if (!isspace(c)) {
            set1.insert(tolower(c));
        }
    }

    for (char c : str1) {
        if (!isspace(c)) {
            set2.insert(tolower(c));
        }
    }

    if (set1 == set2) {
        cout << "Слова являются анаграммами друг друга\n";
    }
    else {
        cout << "Слова не являются анаграммами\n";
    }
```

```
ВЫБОР ЗАДАНИЯ
1. Проверить, являются ли слова анаграммами друг друга
2. Проверьте, есть ли баланс скобок в строке
3. Введите номер в строке и выясните какому слову по счёту в строке он принадлежит

Введите номер задания от 1 до 3 (ESC для выхода):
1
ВЫПОЛНЯЕТСЯ 1 ЗАДАНИЕ
Введите строку1: Hello
Введите строку2: elloh
Слова являются анаграммами друг друга

Введите номер задания от 1 до 3 (ESC для выхода): _
```

Рисунок 1 - Пример выполнения задачи №1

14. Проверьте, имеется ли в заданном тексте баланс открывающих и закрывающих круглых скобок, имея в виду, что балансом, например, будет комбинация (…), в то время как комбинация)..(..)..(балансом не является.

```
void task2() {
    cout << "ВЫПОЛНЯЕТСЯ 2 ЗАДАНИЕ\n";
    string text;
    cout << "Введите текст для проверки баланса скобок: ";
    getline(cin, text);

    int balance = 0;
    int result = 1;

    for (int i = 0; i < text.length(); i++) {
        char c = text[i];
        int ascii_code = static_cast<int>(c);

        if (ascii_code == 40) {
            balance++;
```

```
        }
        else if (ascii_code == 41) {
            balance--;
            if (balance < 0) {
                result = 0;
                break;
            }
        }
    }
    if (balance != 0) {
        result = 0;
    }
    cout << "Результат проверки: " << result << endl;
    cout << "(1 – баланс есть, 0 – баланса нет)" << endl;
        }
```

```
ВЫБОР ЗАДАНИЯ
1. Проверить, являются ли слова анаграммами друг друга
2. Проверьте, есть ли баланс скобок в строке
3. Введите номер в строке и выясните какому слову по счёту в строке он принадлежит

Введите номер задания от 1 до 3 (ESC для выхода):
2
ВЫПОЛНЯЕТСЯ 2 ЗАДАНИЕ
Введите текст для проверки баланса скобок: Hello(privet(kwe)
Результат проверки: 0
(1 - баланс есть, 0 - баланса нет)

Введите номер задания от 1 до 3 (ESC для выхода):
```

Рисунок 2 - Пример решения задачи №2

12. Дана строка символов, состоящая из произвольного текста, слова разделены пробелами. Выведите на экран порядковый номер слова, накрывающего k-ю позицию (если на k-ю позицию попадает пробел, то номер предыдущего слова).

```
void task3() {
    cout << "ВЫПОЛНЯЕТСЯ 3 ЗАДАНИЕ\n";
    string str;
        int k;

        cout << "Введите строку: ";
        getline(cin, str);
        cout << "Введите позицию k: ";
        cin >> k;

        int wordNumber = 0;

        for (int i = 0; i < k; i++) {

            if (str[i] != 32 && (i == 0 || str[i - 1] == 32)) {
                wordNumber++;
            }
        }

        if (str[k - 1] == 32) {
            wordNumber--;
        }
```

```
    cout << "Порядковый номер слова: " << wordNumber << endl;
}
```

```
C:\Users\HONOR\source\repos\ОАИП лаба8\x64\Debug\ОАИП лаба8.exe
ВЫБОР ЗАДАНИЯ
1. Проверить, являются ли слова анаграммами друг друга
2. Проверьте, есть ли баланс скобок в строке
3. Введите номер в строке и выясните какому слову по счёту в строке он принадлежит

Введите номер задания от 1 до 3 (ESC для выхода): 3
ВЫПОЛНЯЕТСЯ 3 ЗАДАНИЕ
Введите строку: Hello beautiful world
Введите позицию k: 5
Порядковый номер слова: 1

Введите номер задания от 1 до 3 (ESC для выхода):
```

Рисунок 3 - Пример решения задачи №3

Контрольные вопросы:

1. Дайте краткое понятие множества в контексте программирования.

Множество - структура данных, хранящая уникальные элементы без определённого порядка.

2. Раскройте значение операций над множествами (объединение, пересечение, разность) и их применение.

– Объединение - элементы из обоих множеств

– Пересечение - общие элементы

– Разность - элементы первого множества, отсутствующие во втором

Применение: анализ данных, поиск общих/уникальных элементов, работа с БД.

3. Сформулируйте сущность алгоритмов проверки принадлежности элемента множеству и их особенности.

– Особенности: скорость зависит от реализации (массив - O(n), хэш-таблица - O(1))

– Алгоритмы: линейный поиск, бинарный поиск (для упорядоченных), хэширование

4. Изложите особенности алгоритмов удаления элемента из множества и добавления элемента в множество.

– Добавление: проверка уникальности + вставка

– Удаление: поиск элемента + его удаление

– Особенности: необходимо поддерживать структуру данных

5. Раскройте особенности применения хэш-таблиц для реализации множеств и их эффективность.

– Особенности: элементы хранятся как ключи (значения игнорируются)

– Эффективность: O(1) в среднем для основных операций, O(n) в худшем случае

– Преимущества: высокая скорость, простые алгоритмы